

Exoplanet Database Explorer

Project Report

Ayan Kashif

Reg. No: 240601050

<https://github.com/vvsayann/Exoplanet-Database-Explorer>

Contents

1	Objective	4
2	Data Source	4
3	Database Design	4
4	Data Import	4
5	Analytical Queries	5
5.1	Detection method frequency	5
5.2	Detection method over time	5
5.3	Size distribution	5
5.4	Multi-planet host stars	5
5.5	Discovery distance over time	6
6	Findings	6
7	Visualizations	6
7.1	Detection method distribution	6
7.2	Size distribution	6
7.3	Discovery distance over time	6
8	Tools Used	6
9	Repository	8

List of Tables

1 Schema of the `exoplanets_full` table. 5

1 Objective

This project was undertaken to gain hands-on practice writing analytical SQL against a large, real, and imperfect dataset, moving beyond introductory exercises on small practice tables. A secondary goal was to produce a concrete, presentable artifact for a CV: a PostgreSQL database of confirmed exoplanets, built and queried to answer genuine astronomy questions, and connected to a Python visualization pipeline.

2 Data Source

Data was drawn from the NASA Exoplanet Archive’s Planetary Systems Composite Parameters (`pscomppars`) table — the current, actively maintained table of confirmed exoplanets, containing 6,336 records at the time of download. Sixteen columns were retained, covering planet identity, discovery details, orbital period, radius, and system distance, along with the associated measurement uncertainties and limit flags provided by the archive.

3 Database Design

A PostgreSQL 18 database (`astronomy`) was used, managed through pgAdmin 4. A dedicated table, `exoplanets_full`, was created to hold the imported dataset. Design choices are summarised below:

- `pl_name` was set as the primary key, since planet names are unique identifiers in this dataset.
- `NUMERIC` was used in place of floating-point types for all measured quantities, to avoid rounding artefacts in scientific values.
- Nearly all columns besides `pl_name` and `hostname` were made nullable, reflecting the genuinely incomplete nature of real observational data — for example, planets discovered via direct imaging frequently lack orbital period measurements.

The full schema is listed below.

4 Data Import

The CSV export from the NASA Exoplanet Archive included descriptive comment lines preceding the actual data, which were stripped before import. The cleaned file was then loaded into `exoplanets_full` using pgAdmin’s Import/Export tool, with the header row mapped directly to the corresponding table columns. The import was verified by row count and a manual inspection of the first rows returned.

Table 1: Schema of the `exoplanets_full` table.

Column	Type	Description
<code>pl_name</code>	<code>VARCHAR(100)</code> PK	Planet name
<code>hostname</code>	<code>VARCHAR(100)</code>	Host star name
<code>discoverymethod</code>	<code>VARCHAR(50)</code>	Detection method
<code>disc_year</code>	<code>INTEGER</code>	Year of discovery
<code>pl_controv_flag</code>	<code>SMALLINT</code>	Controversial-detection flag
<code>pl_orbper</code>	<code>NUMERIC</code>	Orbital period (days)
<code>pl_orbpererr1 / err2</code>	<code>NUMERIC</code>	Orbital period upper / lower uncertainty
<code>pl_orbperlim</code>	<code>SMALLINT</code>	Orbital period limit flag
<code>pl_rade</code>	<code>NUMERIC</code>	Planet radius (Earth radii)
<code>pl_radeerr1 / err2</code>	<code>NUMERIC</code>	Radius upper / lower uncertainty
<code>pl_radelim</code>	<code>SMALLINT</code>	Radius limit flag
<code>sy_dist</code>	<code>NUMERIC</code>	System distance (parsecs)
<code>sy_disterr1 / err2</code>	<code>NUMERIC</code>	Distance upper / lower uncertainty

5 Analytical Queries

Five queries were written to answer specific astronomical questions about the dataset, introducing `CASE WHEN`, `HAVING`, and `AVG()` alongside previously learned SQL fundamentals (`SELECT`, `WHERE`, `ORDER BY`, `JOIN`, `GROUP BY`, `COUNT`).

5.1 Detection method frequency

Grouping and counting planets by `discoverymethod` shows Transit as the dominant detection technique, followed by Radial Velocity, with a long tail of less common methods.

5.2 Detection method over time

Grouping by both `disc_year` and `discoverymethod` shows Radial Velocity as the sole productive method through the early 2000s, with Transit detections beginning around 2002 and becoming dominant following Kepler’s 2009 launch.

5.3 Size distribution

Planets were bucketed into four size categories by radius (Earth-sized, Super-Earth, Neptune-sized, Jupiter-sized) using a `CASE WHEN` expression, then counted per category.

5.4 Multi-planet host stars

Grouping by `hostname` with a `HAVING COUNT(*) > 1` filter isolates genuine multi-planet systems, sorted by planet count.

5.5 Discovery distance over time

Grouping by `disc_year` and taking `AVG(sy_dist)` shows how the typical distance of newly discovered planets has changed as surveys have advanced, alongside the count of planets found each year for context.

6 Findings

- **Detection method shifted over time.** Radial Velocity dominated discoveries through the early 2000s, but Transit photometry overtook it after Kepler’s 2009 launch and now accounts for the large majority of confirmed planets (4,676 of 6,336).
- **Confirmed planets skew large.** Neptune-sized and Jupiter-sized planets outnumber Earth-sized ones by roughly 4:1 — not because small planets are rarer in the galaxy, but because Transit and Radial Velocity are both far more sensitive to large planets, which block more light or cause larger stellar wobbles.
- **Known multi-planet systems appear correctly at the top of the ranking.** The planets-per-star query surfaces KOI-351 (Kepler-90, 8 planets) and TRAPPIST-1 (7 planets), a useful check that the data and query logic behave as expected.
- **Discovery distance tracks survey history.** Early Radial-Velocity-era planets (1995–2003) cluster within roughly 50 parsecs, since Radial Velocity only works well on bright, nearby stars. A large batch of Kepler-confirmed planets around 2016 sits much farther out (roughly 800 parsecs), reflecting Kepler’s single distant field of view. TESS-era planets (2018 onward) hold a similarly high, steady distance with more consistent year-to-year counts.

7 Visualizations

A Python pipeline (SQLAlchemy, pandas, matplotlib) was built to connect to the database, run the analytical queries directly, and plot the results, closing the loop from raw data to finished visualization without manual export steps.

7.1 Detection method distribution

7.2 Size distribution

7.3 Discovery distance over time

8 Tools Used

- PostgreSQL 18 and pgAdmin 4 for database design, import, and querying.
- NASA Exoplanet Archive as the data source.

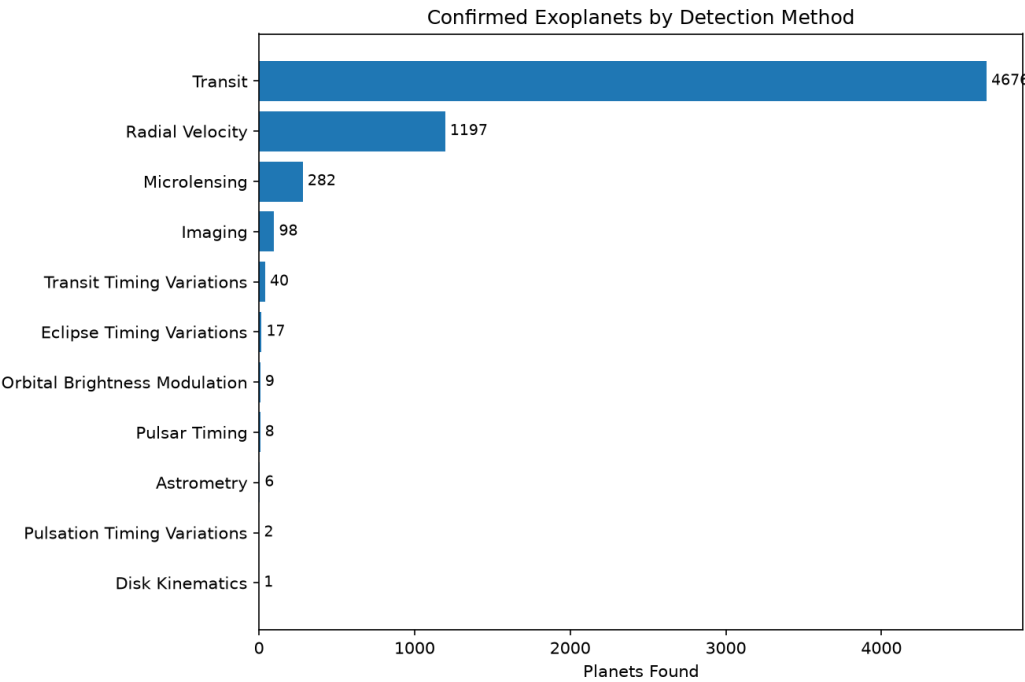


Figure 1: Confirmed exoplanets by detection method.

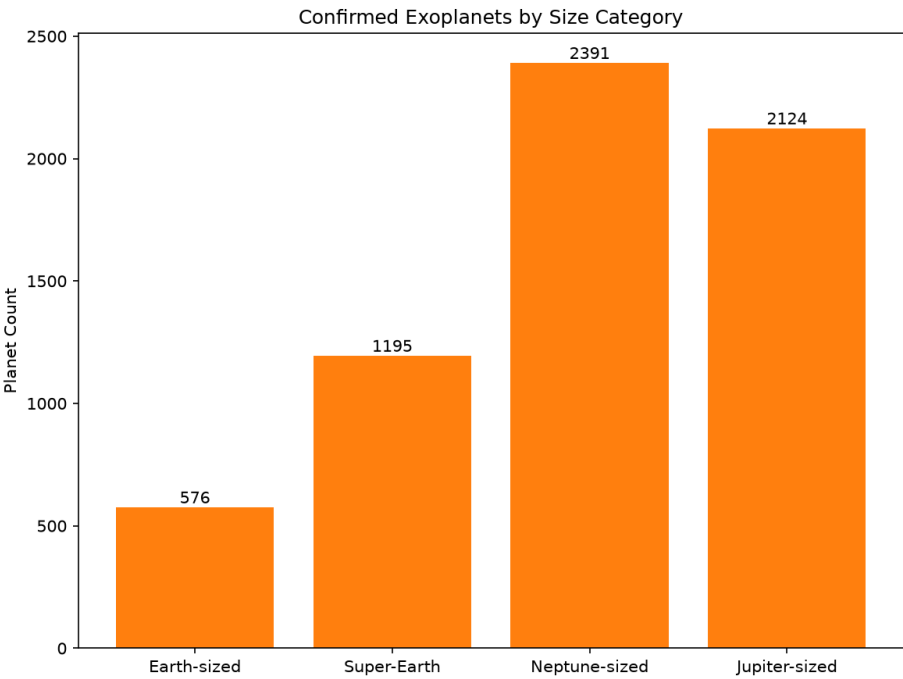


Figure 2: Confirmed exoplanets by size category.

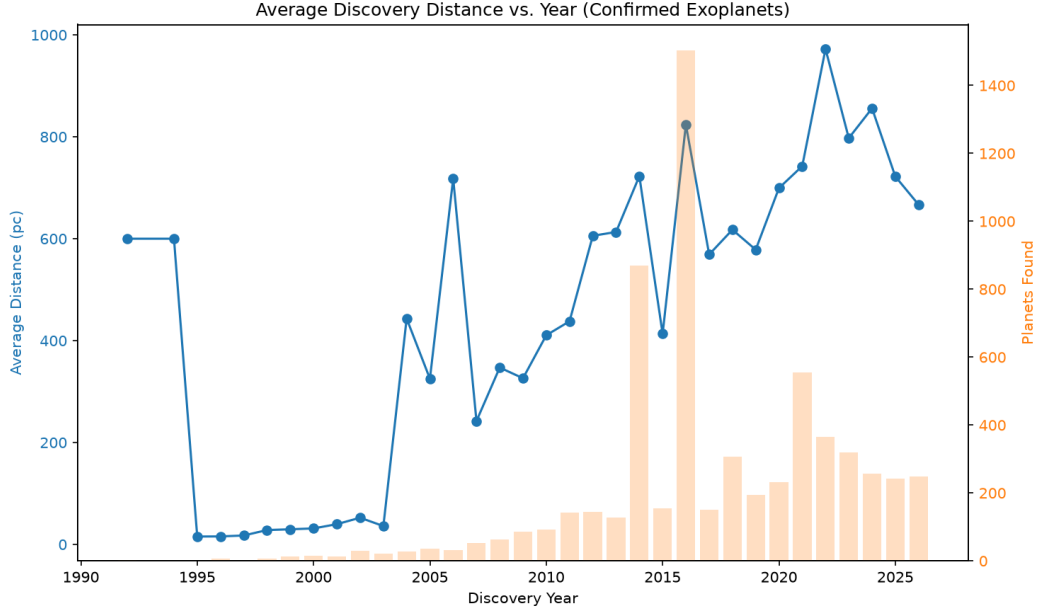


Figure 3: Average discovery distance per year, with annual planet count shown as bars.

- Python (pandas, SQLAlchemy, matplotlib) for querying the database programmatically and producing visualizations.
- Git and GitHub for version control and publishing the finished project.

9 Repository

The complete project — schema, queries, Python scripts, and README — is published at:

<https://github.com/vvsayann/Exoplanet-Database-Explorer>